

Lecture 2: Communication (Remote Procedure Calls – RPC)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2025

Today's Topic: Communication

Implementing Remote Procedure Calls

Andrew D. Birrell and Bruce Jay Nelson, ACM Trans. Comput. Syst., 2(1):39-59, 1984.

SIGOPS Hall of Fame Award

Optional:

A Critique of the Remote Procedure Call

Andrew S. Tanenbaum and Robbert van Renesse, 1988

Why Today's Paper?

- Communication essential for distributed systems
- Popular, fundamental paradigm for communication
- RPC still used, many different implementations, project
- Detailed design and implementation choices
 - How do to binding/naming
 - Handling errors: Server and client crashes; lost and duplicate messages
 - Semantics (at-most-once vs. at-least-once)

Goals

Communication Alternatives?

1)

Pros and cons?

2)

Alternative: Distributed Shared Memory (DSM)

Problems?

Virtual address space



page tables



1) local memory

2) local disk

3)



local DRAM

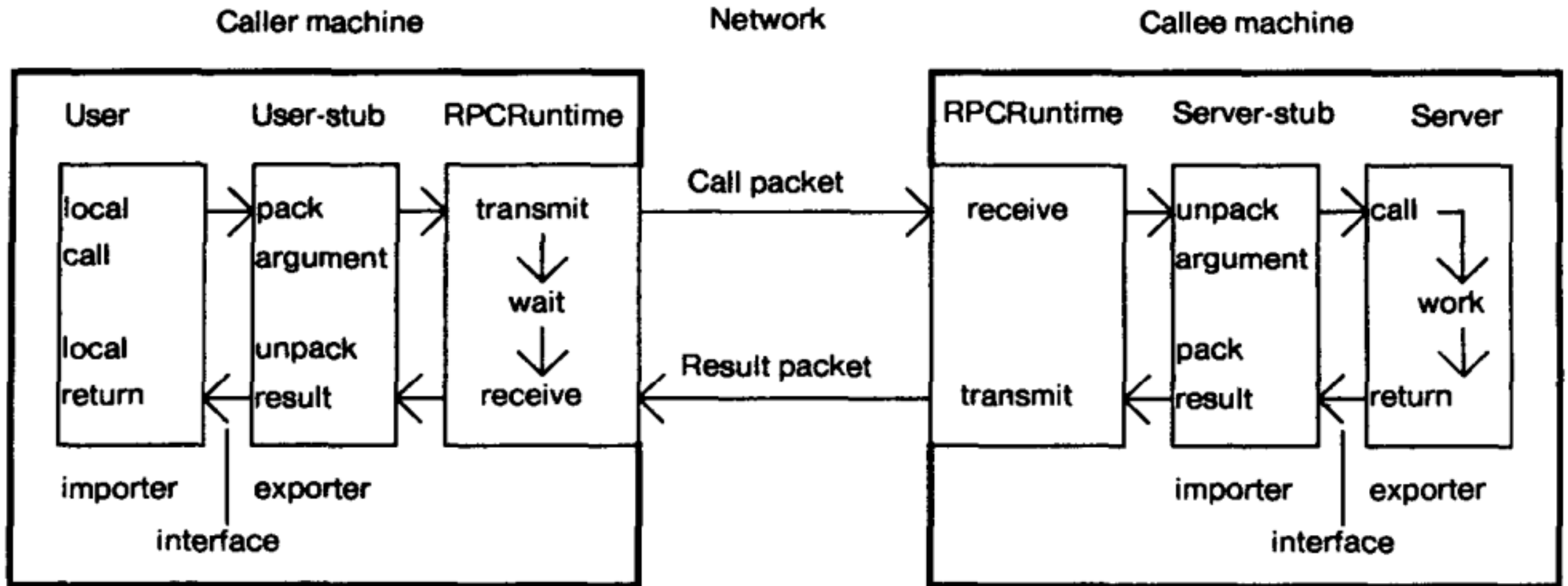
Physical mem.



Communication Comparison

	Performance	Ease of Use	Generality	Handle Failures
Raw Messages				
DSM				
RPC				

RPC Basics



What does stub library layer look like?

Client: `rc = foo(5);`

Client stub:

```
int foo(int arg) {  
    m = msg_create();  
    pack_int(m, arg);  
    send_msg(DEST, m);  
    m2 = wait_response();  
    rc = unpack_int(m2);  
    return rc;  
}
```

Advantages?

Generality Challenge: How to pass/deference pointer args?

`int bar (int *ptr)`

What does original RPC specification do?

Other options?

Problems?

Binding: How to identify server?

Client c: Wants to use KVstore service with API get() and put()

Leverage Grapevine for naming: distributed replicated database

KVstore	a, b
a	1.2
b	1.3

Server d: export ("Kvstore", my-kv-dispatcher)

Grapevine: check d is valid Kvstore; add d

What is d's my-kv-dispatcher? Why?

What state does Server d maintain?

Export Table

Service	Dispatcher	Unique Service Id
Lock-service	My-lock-dispatch	2837519

How to set unique service id?

Why is unique service id important?

Client c: Import("Kvstore")

c->grapevine: ?

c->d ("Kvstore"): <?,?>

c->d ("get"): "call packet" sends...

Transport Layer

Goals?

Assumptions?

Semantics of RPC calls

1) Local machine procedure call guarantee:

2) At-Least-Once:

When ok?

Examples?

Why not okay for RPC?

3) **RPC:**

At-most-once Semantics

General idea?

How many responses must server remember?

Implications?

At-most-Once Details: Acknowledgements

C->s: call packet {call-id, procedure, args}

S->c: {call-id, results}

How is procedure specified?

How can client know server received call?

Naïve:

Optimization:

How can server know client received result?

Naïve:

Optimization:

Assumption?

At-most-Once Details: Call-id

C->s: call packet {call-id, procedure, args}

S->c: {call-id, results}

call-id: {[calling machine id, process], sequence number}

Why call-id (sequence number) in result?

Why call-id (sequence number) in call packet?

Call Table on Server

Activity	Last Sequence #	Last Result
{machine id, PID}	L	R

Msg arrives with seq # I
i > L ?
i = L ?
i < L ?

State per Connection?

Important to minimize state on servers in distributed systems

How much state per connection on server?

Server: Export Table + Call Table

Service	Dispatcher	Unique Service Id	Activity	Last Sequence #	Last Result
Lock-service	My-lock-dispatch	2837519	{machine id, PID}	L	R

Client: Last seq # (per ?); sequential or monotonic?

What happens on Failures?

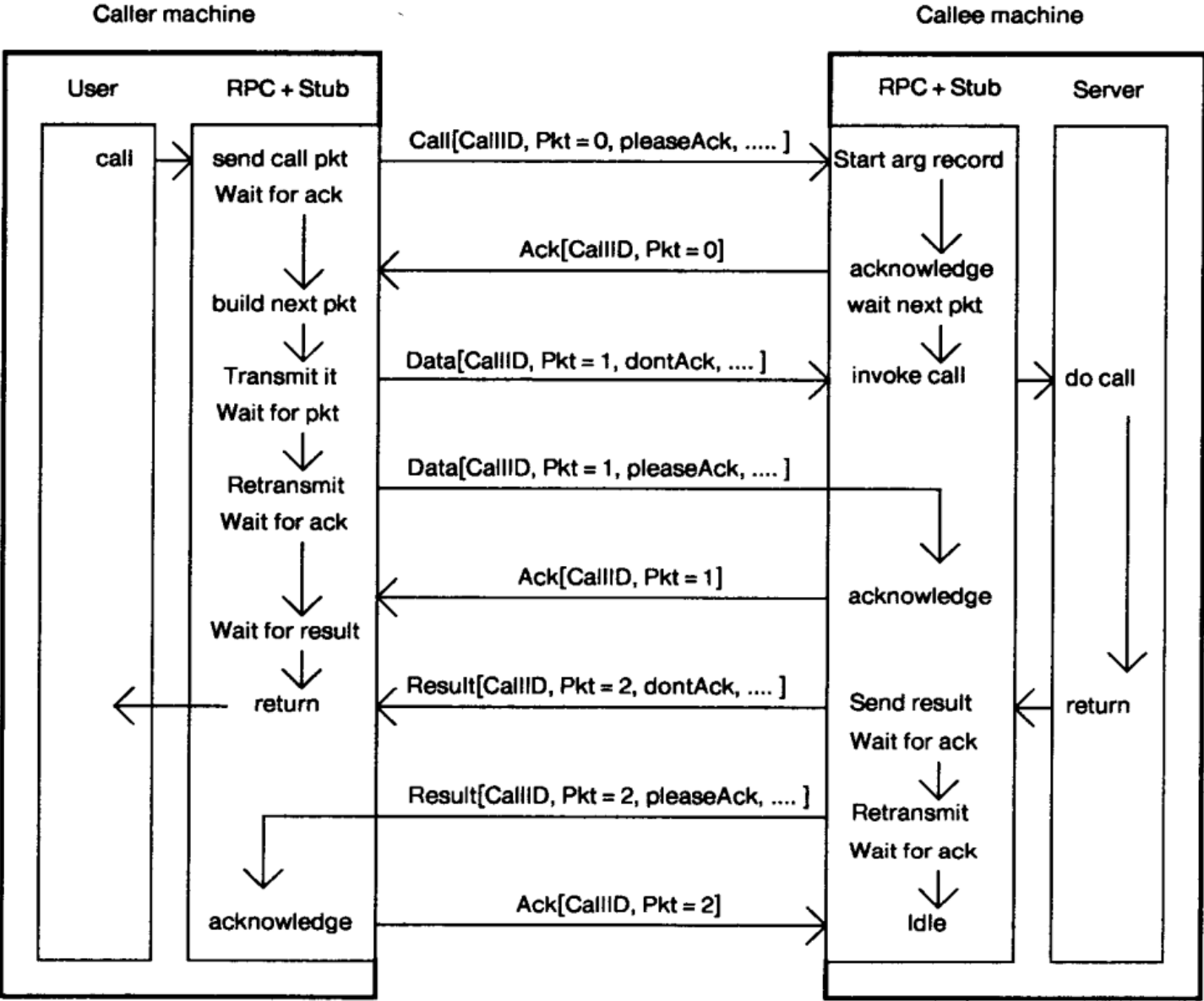
Server crash:

- Lose contents of Export Table, reboot, start up services again
- New Unique Service ID → Breaks binding with client
- Client informed on next interaction, or ??
- **What happens to last RPC before crash?**
 - Maybe exec, maybe not → Notify client
 - “At-most once” (not exactly once)

Client crash:

- **What happens to ongoing RPCs?**
- Reboot, **What is next sequence number?**
- **How does server respond?**

Complicated Calls: Large Args w/ Multiple Packets



Implications?

Evaluation

Did they show they met their goals?

Ease-of-use

Highly efficient

Powerful semantics

Secure communication (turn off encryption)

Hide failures

Generality

Procedure	Minimum	Median	Transmission	Local-only
no args/results	1059	1097	131	9
1 arg/result	1070	1105	142	10
2 args/results	1077	1127	152	11
4 args/results	1115	1171	174	12
10 args/results	1222	1278	239	17
1 word array	1069	1111	131	10
4 word array	1106	1153	174	13
10 word array	1214	1250	239	16
40 word array	1643	1695	566	51
100 word array	2915	2926	1219	98
resume except'n	2555	2637	284	134

Conclusions

RPC still very popular programming paradigm

- Assumes client/server, not bulk data xfer, only 1 outstanding request/client
- Can be built on other messaging protocols (UDP or TCP)

Leverage Grapevine for naming: distributed replicated database

RPC: At-most-once semantics

- Easier if could assume operations are **idempotent (at-least-once)**

Minimal state on server

- Track last seq-id + result (per activity)

Unique service id useful

- Identify server restarted

Sequence numbers useful

- Match responses/requests
- Identify duplicate requests (save last result)

Announcements

Reading Group 1: Meet with some members of reading group by Sunday

- Submit short individual write-up in Canvas
- Discuss past or future papers, could go over posted questions
- Will meet with same group for Week 2

Next: Failures

- Jim Gray. Why Do Computers Stop And What Can Be Done About It?
HP Labs Technical Report TR-85.7 **SIGOPS Hall of Fame Award**
- Aishwarya Ganesan, Ramnatthan Alagappan,
Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau,
Redundancy Does Not Imply Fault Tolerance: Analysis of Distributed Storage
Reactions to Single Errors and Corruptions,
Proc. of 15th USENIX Conf. on File and Storage Technologies (FAST '17)